operator. Like most of the functions in the standard library that take functions or predicates as arguments, `std::call_once` works with any function or callable object.

```
std::shared_ptr<some_resource> resource_ptr;
std::once_flag resource_flag;                          <-(1)

void init_resource()
{
    resource_ptr.reset(new some_resource);
}

void foo()
{
    std::call_once(resource_flag,init_resource);       <- Initialization is called exactly once
    resource_ptr->do_something();
}
```

In this example, both the `std::once_flag` (1) and data being initialized are namespace-scope objects, but `std::call_once()` can just as easily be used for lazy initialization of class members, as in the following listing.

**Listing 3.12. Thread-safe lazy initialization of a class member using `std::call_once`**